

Cut

Curso Programación Lógica 2024

Ejemplo teórico

$p(X) \text{ :- } a(X).$

$p(X) \text{ :- } b(X), c(X), d(X), e(X).$

$p(X) \text{ :- } f(X).$

$a(1).$

$b(1).$

$c(1).$

$b(2).$

$c(2).$

$d(2).$

$e(2).$

$f(3).$

$p(X) \vee \neg a(X)$

$p(X) \vee \neg b(X) \vee \neg c(X) \vee \neg d(X) \vee \neg e(X)$

$p(X) \vee \neg f(X)$

$a(1)$

$b(1)$

$c(1).$

$b(2).$

$c(2).$

$d(2).$

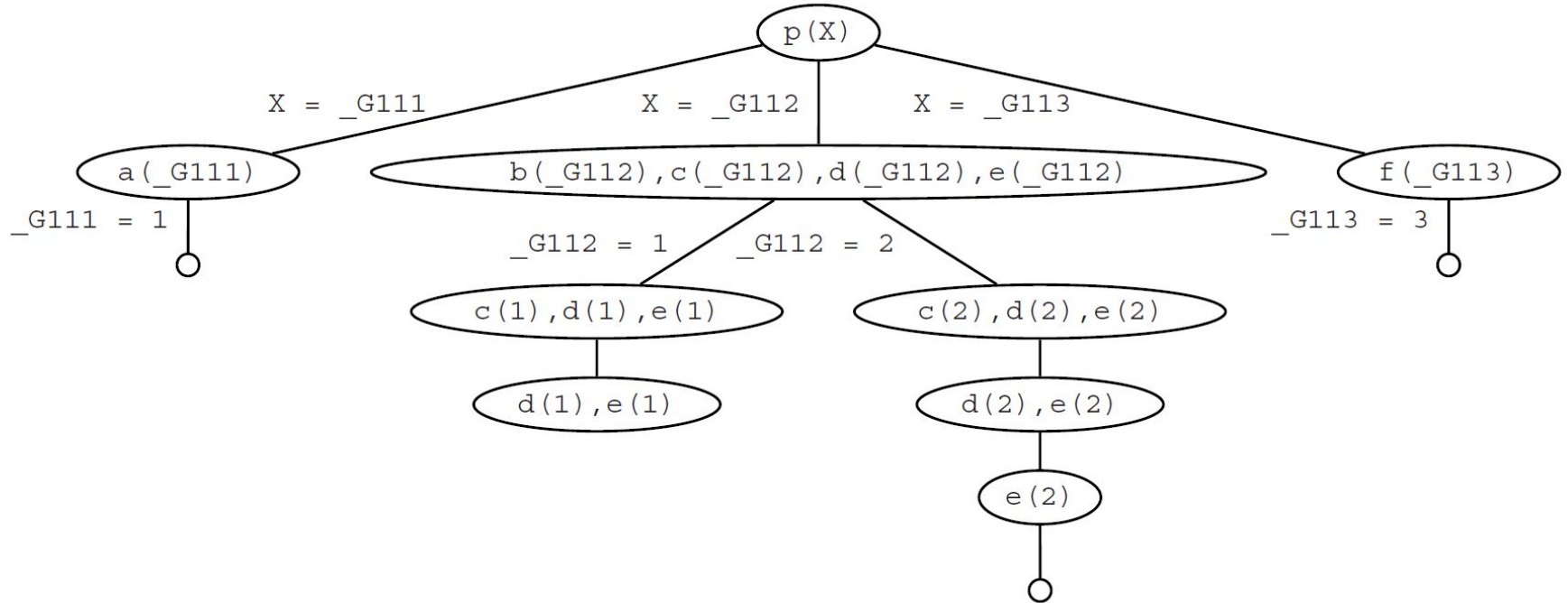
$e(2).$

$f(3).$

Refutación SLD (brevísimo repaso)

- Quiero saber si $\exists x / p(x)$ [Esto no es una [cláusula de Horn](#)]
 - (Equivalentemente) Quiero saber si $\forall x / \neg p(x)$ es F [Esto sí lo es!]
 - Lo que es lo mismo... $\leftarrow p(x)$ [Objetivo]
 - Una refutación SLD para $P \cup \{G\}$ es una derivación SLD finita para $P \cup \{G\}$ cuyo último objetivo es $\square$
-
- Paso 1: Si $\leftarrow p(X)$ es Falso, como sé que $p(x) \leftarrow a(X)$ (o, lo que es lo mismo, $p(X) \vee \neg a(X)$) es verdadero, entonces $\neg a(X)$ (o, lo que es lo mismo, $\leftarrow a(X)$) es Falso.
 - ... Y así sigo con SLD hasta llegar a $\square$

Ejemplo



cut

*The goal **succeeds** and **commits Prolog** to the choices made since the parent goal was unified with the head of the clause the cut occurs in . (The Art of Prolog)*

Ejemplo teórico

$p(X) \text{ :- } a(X).$

$p(X) \text{ :- } b(X), c(X), \textcolor{red}{!}, d(X), e(X).$

$p(X) \text{ :- } f(X).$

$a(1).$

$b(1).$

$c(1).$

$b(2).$

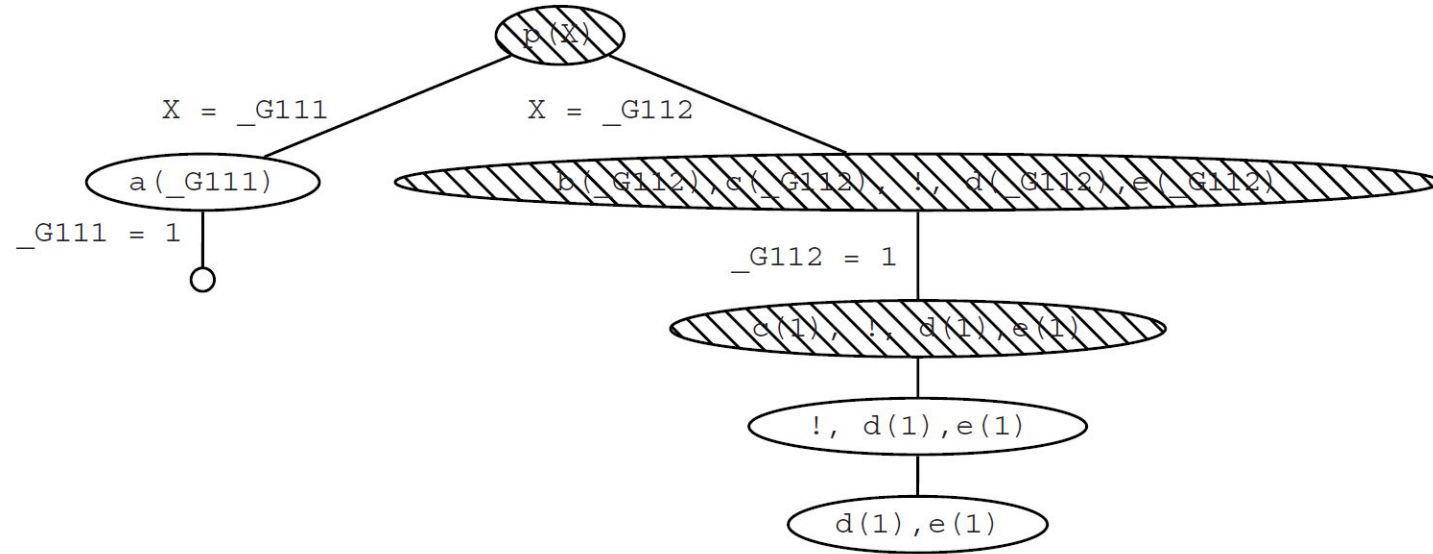
$c(2).$

$d(2).$

$e(2).$

$f(3).$

Ejemplo



Ejemplo 2

$s(X,Y) \text{ :- } q(X,Y).$

$s(0,0).$

$q(X,Y) \text{ :- } i(X),j(Y).$

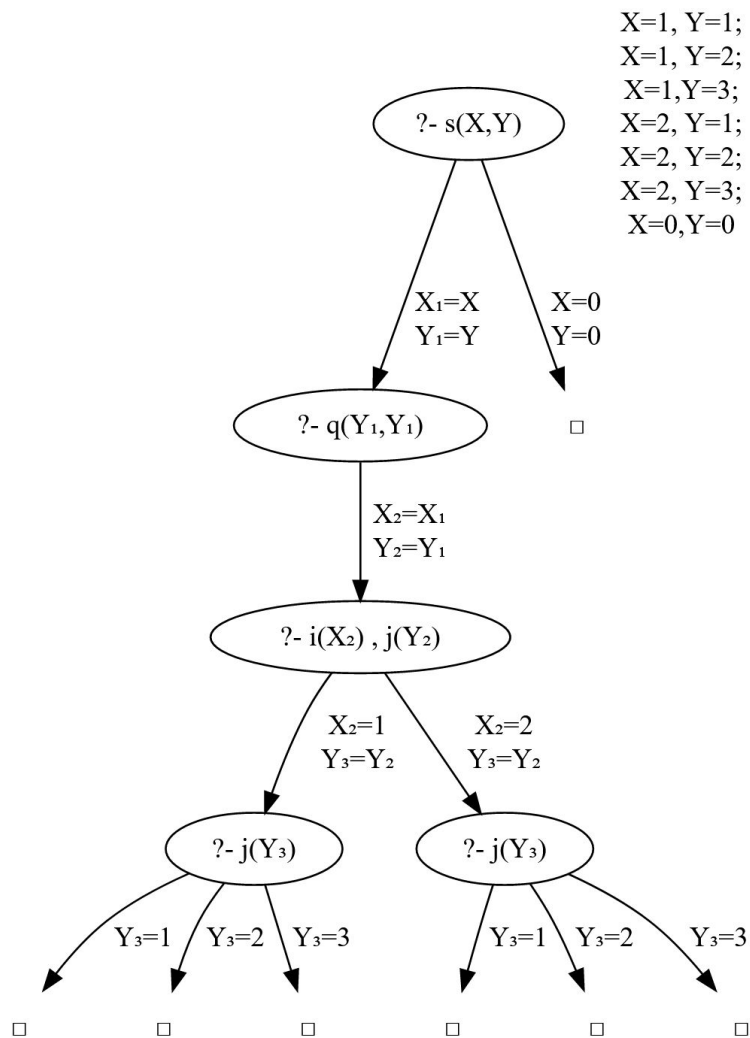
$i(1).$

$i(2).$

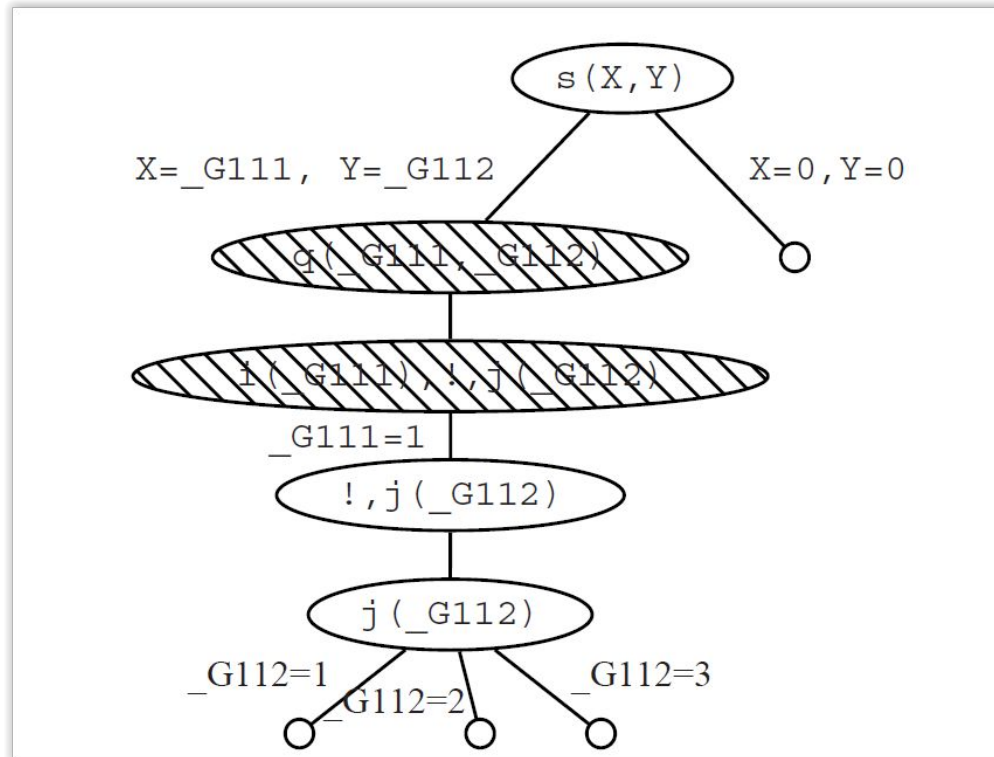
$j(1).$

$j(2).$

$j(3).$



Ejemplo 2



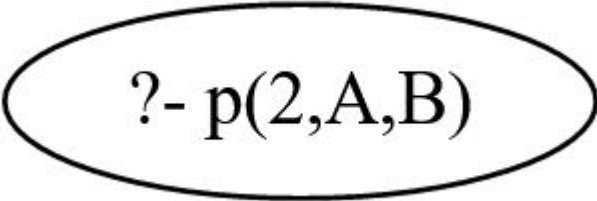
Ejercicio 2 Práctico 7

```
ej2_p(X,Y,Z):-  
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-  
    N is X + 1,  
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-  
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-  
    N is X - 1,  
    ej2_q(N,Y,Z).
```

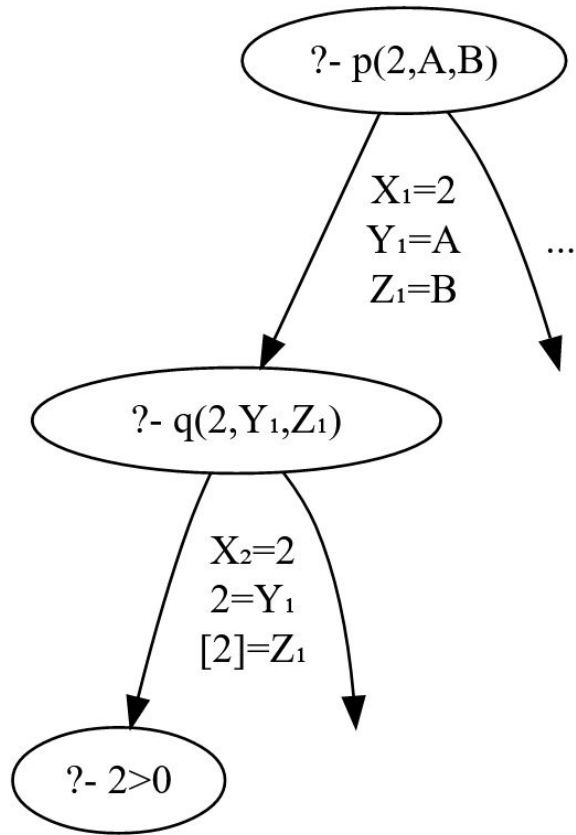
?- p(2,A,B)

```
ej2_p(X,Y,Z):-  
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-  
    N is X + 1,  
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-  
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-  
    N is X - 1,  
    ej2_q(N,Y,Z).
```

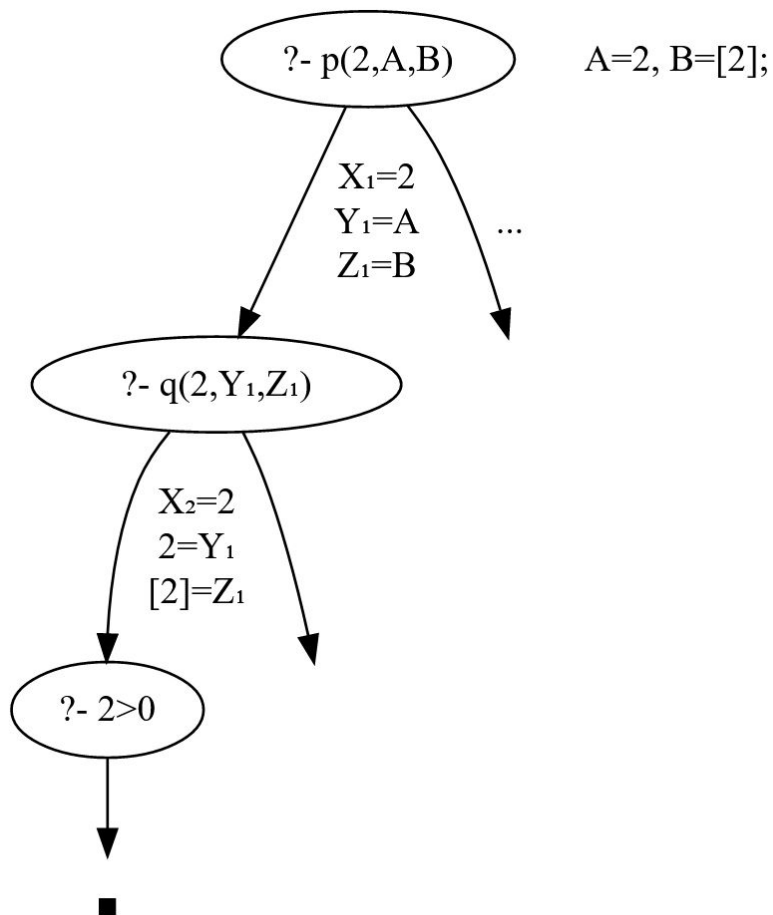


```
ej2_p(X,Y,Z):-
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-
    N is X + 1,
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-
    N is X - 1,
    ej2_q(N,Y,Z).
```

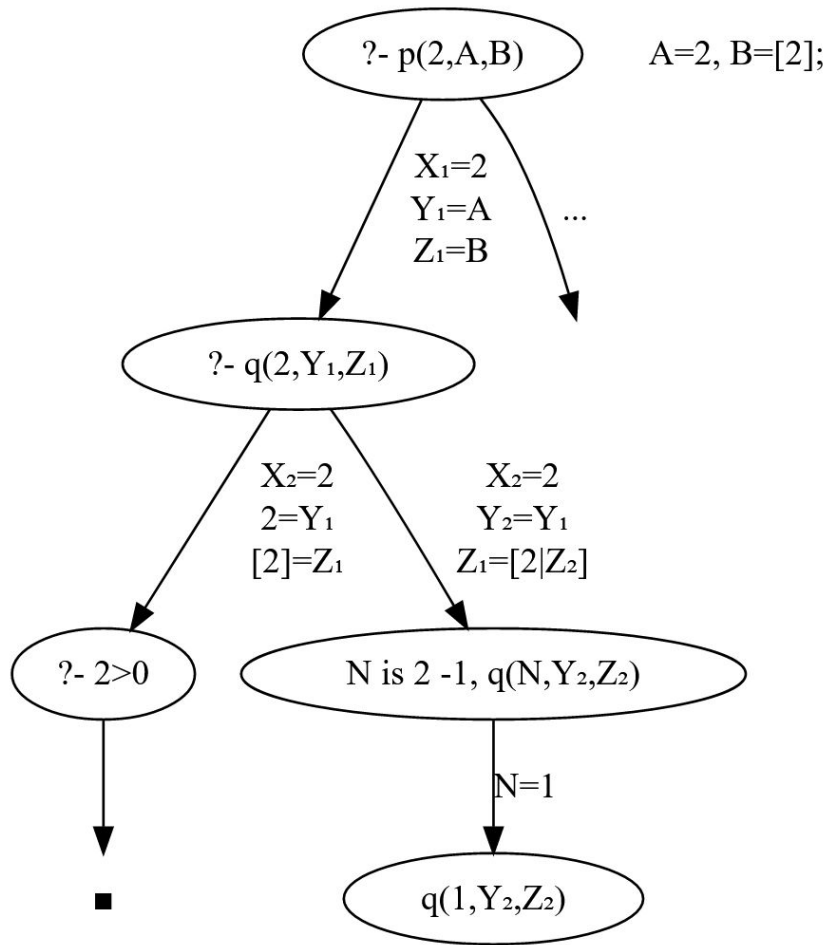


```
ej2_p(X,Y,Z):-
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-
    N is X + 1,
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-
    N is X - 1,
    ej2_q(N,Y,Z).
```



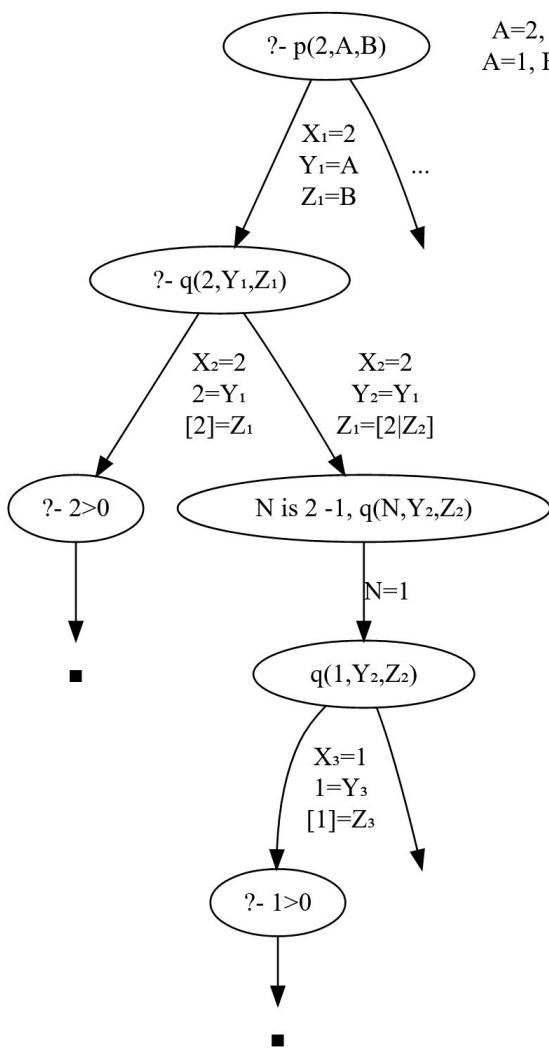
```
ej2_p(X,Y,Z):-
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-
    N is X + 1,
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-
    N is X - 1,
    ej2_q(N,Y,Z).
```

A=2, B=[2];
A=1, B=[2,1];

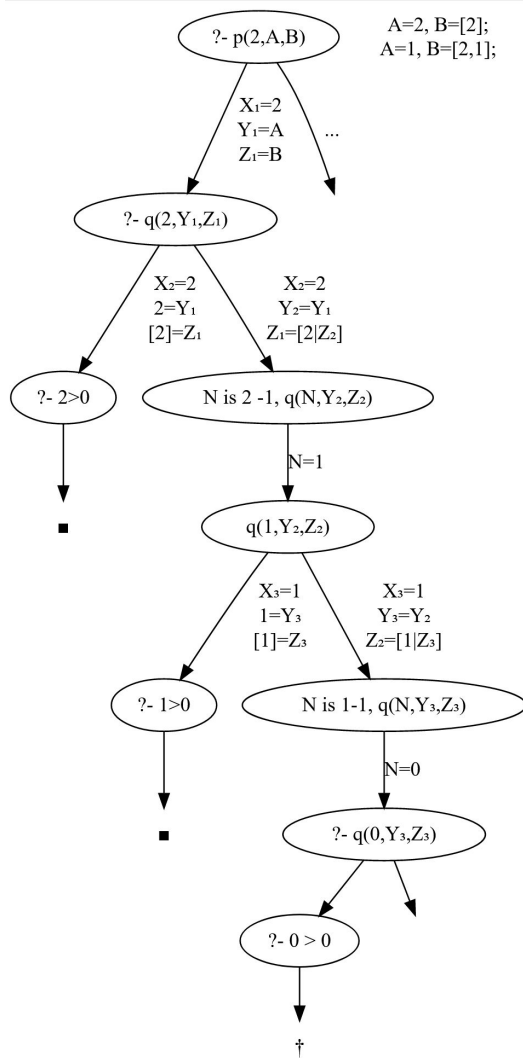


```
ej2_p(X,Y,Z):-
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-
    N is X + 1,
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-
    N is X - 1,
    ej2_q(N,Y,Z).
```



```
ej2_p(X,Y,Z):-
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-
    N is X + 1,
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-
    X>0.
```

```
ej2_q(X,Y,[X|Z]):-
    N is X - 1,
    ej2_q(N,Y,Z).
```


¿Qué pasa si agregamos un cut en q?

```
ej2_p(X,Y,Z):-  
    ej2_q(X,Y,Z).
```

```
ej2_p(X,Y,Z):-  
    N is X + 1,  
    ej2_p(N,Y,Z).
```

```
ej2_q(X,X,[X]):-  
    X>0,!.
```

```
ej2_q(X,Y,[X|Z]):-  
    N is X - 1,  
    ej2_q(N,Y,Z).
```

Ejercicio 5

`par(X):- 0 is (X mod 2).`

`multiplo_tres(X) :- 0 is (X mod 3).`

`sel_par([X|_],X,Z) :- par(X), X>Z.`

`sel_par([_|Xs],Y,Z) :- sel_par(Xs,Y,Z).`

`prob(X):- sel_par([1,4,14,5,32,30,17,18],X,10), multiplo_tres(X).`

Ejercicio 5

`par(X):- 0 is (X mod 2).`

`multiplo_tres(X) :- 0 is (X mod 3).`

`sel_par([X|_],X,Z) :- par(X), X>Z, !.`

`sel_par([_|Xs],Y,Z) :- sel_par(Xs,Y,Z).`

`prob(X):- sel_par([1,4,14,5,32,30,17,18],X,10), multiplo_tres(X).`

Ejercicio 5

`par(X):- 0 is (X mod 2).`

`multiplo_tres(X) :- 0 is (X mod 3).`

`sel_par([X|_],X,Z) :- par(X), X>Z.`

`sel_par([_|Xs],Y,Z) :- sel_par(Xs,Y,Z).`

`prob(X):- sel_par([1,4,14,5,32,30,17,18],X,10), !,
multiplo_tres(X).`

Ejercicio 5

`par(X):- 0 is (X mod 2).`

`multiplo_tres(X) :- 0 is (X mod 3).`

`sel_par([X|_],X,Z) :- par(X), X>Z.`

`sel_par([_|Xs],Y,Z) :- sel_par(Xs,Y,Z).`

`prob(X):- sel_par([1,4,14,5,32,30,17,18],X,10), multiplo_tres(X),
!,.`

¿Para qué sirve? (cut verde)

max/3

`max(X,Y,Y) :- X =< Y.`

`max(X,Y,X) :- X>Y.`

max/3

`max(X,Y,Y) :- X =< Y, !.`

`max(X,Y,X) :- X>Y.`

Pero hay que tener cuidado (cut rojo)

max/3

`max(X,Y,Y):- X =< Y, !.`

`max(X,Y,X).`

¿Qué pasa con `max(200,300,200)`?

Negación por falla

not (o, mejor, fail if)

- Si $\text{not}(X)$ se satisface, quiere decir que X no se puede deducir de nuestra base de conocimientos. Esto es completamente distinto a que X no sea verdadero.
- Debido a la presencia de ramas infinitas en los árboles de derivación, no voy a poder afirmar que algo no se puede demostrar con los hechos que tengo. La regla de negación por falla finita dice que si NINGUNA de las ramas del árbol me conduce a una refutación, puedo negar el objetivo.

not (o, mejor, fail if)

```
fail_if(Goal):- Goal,!,fail.
```

```
fail_if(Goal).
```

not (o, mejor, fail if)

```
not_member_check(X,Xs) :- not(member(X,Xs)).
```

```
?- not_member_check(3,[1,2]).
```

not (o, mejor, fail if)

```
not_member_check(X,Xs) :- not(member(X,Xs)).
```

```
?- not_member_check(3,[1,2]).
```

```
true.
```

not (o, mejor, fail if)

```
disjuntos(Xs,Ys) :- not(member(X,Xs)),not(member(X,Ys)).
```

```
?- disjuntos([1,2,3],[4,5]).
```

not (o, mejor, fail if)

```
disjuntos(Xs,Ys) :- not(member(X,Xs)),not(member(X,Ys)).
```

```
?- disjuntos([1,2,3],[4,5,6]).
```

```
false.
```

not (o, mejor, fail if)

pareja(juan,ana).

pareja(X,Y) :- pareja(Y,X).

? pareja(juan,ana)

true;...

? \+ pareja(juan,ana)

false

not (o, mejor, fail if)

```
estudiante_soltero(X) :-  
    \+ casado(X), estudiante(X)
```

```
estudiante(juan).
```

```
casado(pedro).
```

```
? estudiante_soltero(X)
```


not (o, mejor, fail if)

```
estudiante_soltero(X) :-  
    \+ casado(X), estudiante(X)
```

```
estudiante(juan).
```

```
casado(pedro).
```

```
? estudiante_soltero(X)
```

```
false
```

not (o, mejor, fail if)

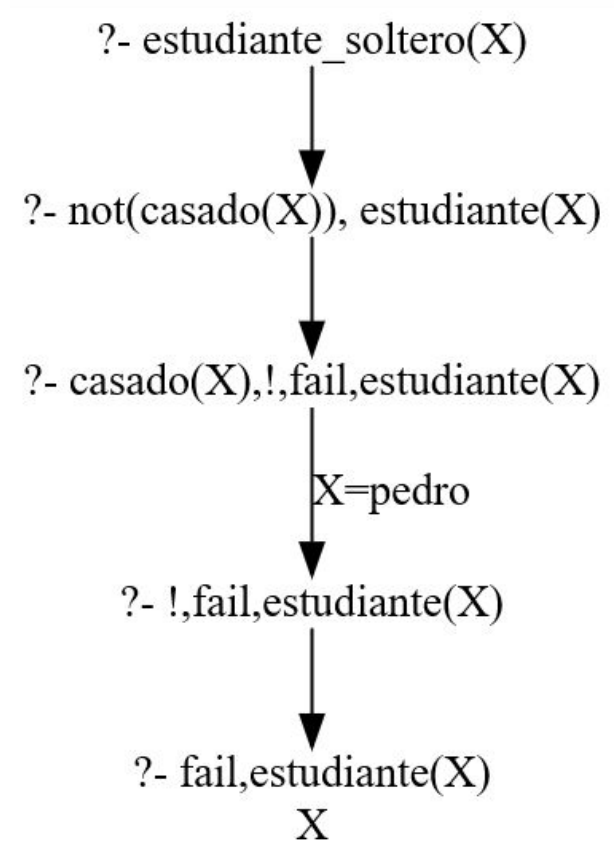
```
estudiante_soltero(X) :-  
    \+ casado(X), estudiante(X)
```

```
estudiante(juan).
```

```
casado(pedro).
```

```
? estudiante_soltero(X)
```

```
false
```



not (o, mejor, fail if)

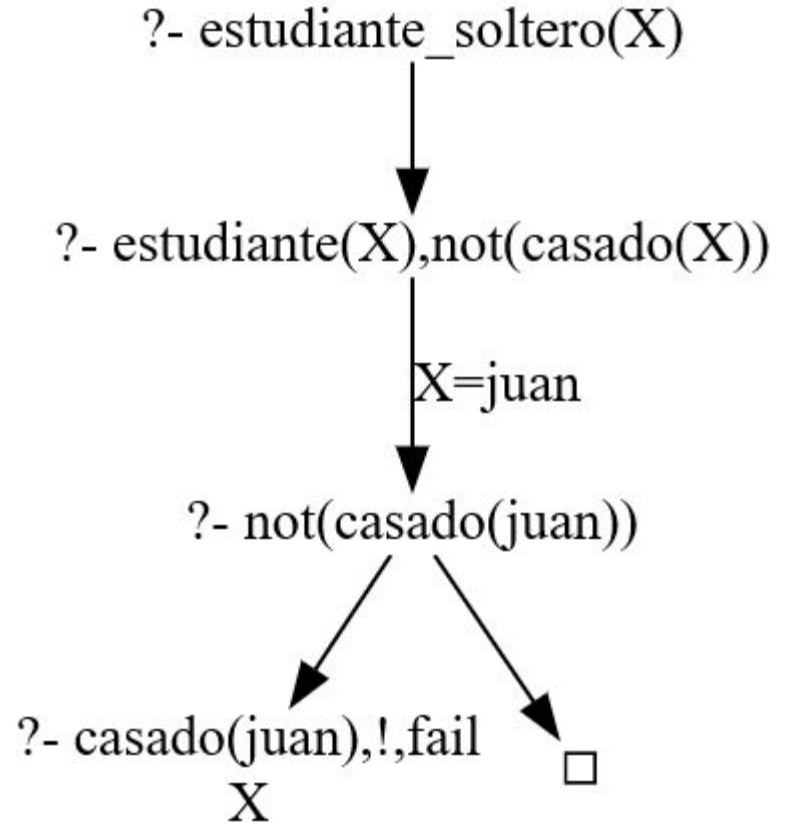
```
estudiante_soltero(X) :-  
    estudiante(X),  
    \+ casado(X).
```

```
estudiante(juan).
```

```
casado(pedro).
```

```
? estudiante_soltero(X)
```

```
true
```



not (o, mejor, fail if)

Regla general: not puede causarnos problemas si los objetivos a negar no están completamente instanciados.

Ejercicio 8 ¿Grafo completo?

vertice(g,a).

vertice(g,b).

vertice(g,c).

vertice(g,d).

arista(g,a,b).

arista(g,b,c).

arista(g,c,d).

arista(g,d,a).

arista(g,b,d).

Ejercicio 8 ¿Grafo completo?

% Completo es que no es incompleto. Incompleto es si hay un par de vértices que no tienen aristas que los comuniquen.

Ejercicio 8 ¿Grafo completo?

% Completo es que no es incompleto. Incompleto es si hay un par de vértices que no tienen aristas que los comuniquen.

```
incompleto(G):-  
    vertice(G,V1),  
    vertice(G,V2),  
    V1 \= V2,  
    not(arista(G,V1,V2)).
```

Ejercicio 8 ¿Grafo completo?

% Completo es que no es incompleto. Incompleto es si hay un par de vértices que no tienen aristas que los comuniquen.

```
incompleto(G):-  
    vertice(G,V1),  
    vertice(G,V2),  
    V1 \= V2,  
    not(arista(G,V1,V2)).
```

```
completo(G):- not(incompleto(G)).
```


Referencias

- Capítulo 11 The Art of Prolog
- Capítulo 10 Learn Prolog Now!